Supersetid :8013480

Digital Nurture 5.0 Deep Skilling Handbook Java FSE - Solution

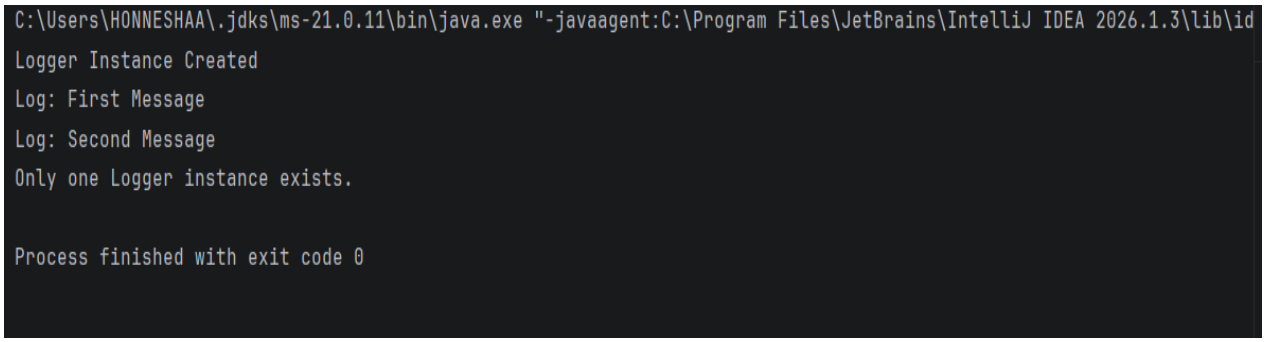
DESIGN PATTERNS AND PRINCIPLES

Supersetid :8013480

Exercise 1: Implementing the Singleton Pattern

```
public class SingletonTest {
    static class Logger {
        private static Logger instance;
        private Logger() {
            System.out.println("Logger Instance Created");
        }
        public static Logger getInstance() {
            if (instance == null) {
                instance = new Logger();
            }
            return instance;
        }
        public void log(String message) {
            System.out.println("Log: " + message);
        }
    }
    public static void main(String[] args) {
        Logger logger1 = Logger.getInstance();
        Logger logger2 = Logger.getInstance();
        logger1.log("First Message");
        logger2.log("Second Message");
        if (logger1 == logger2) {
            System.out.println("Only one Logger instance exists.");
        } else {
            System.out.println("Multiple instances exist.");
        }
    }
}
```

OUTPUT:



Supersetid :8013480

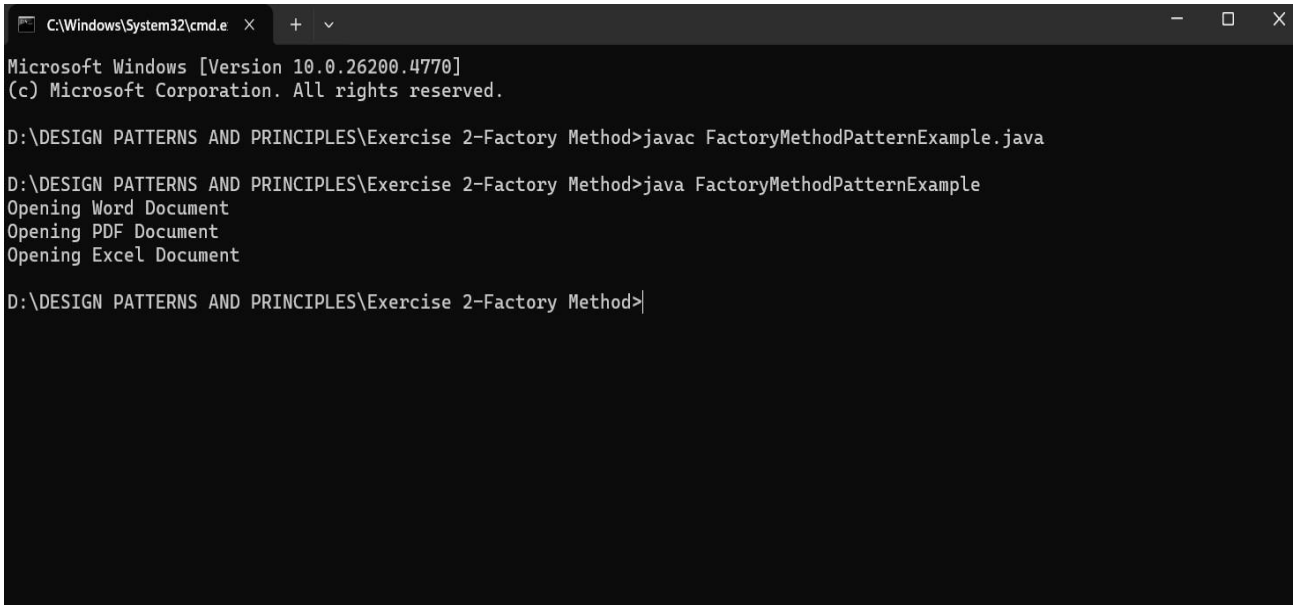
Exercise 2: Implementing the Factory Method Pattern

```
public class FactoryMethodPatternExample {
    interface Document {
        void open();
    }
    static class WordDocument implements Document {
        public void open() {
            System.out.println("Opening Word Document");
        }
    }
    static class PdfDocument implements Document {
        public void open() {
            System.out.println("Opening PDF Document");
        }
    }
    static class ExcelDocument implements Document {
        public void open() {
            System.out.println("Opening Excel Document");
        }
    }
    static abstract class DocumentFactory {
        abstract Document createDocument();
    }
    static class WordFactory extends DocumentFactory {
        Document createDocument() {
            return new WordDocument();
        }
    }
    static class PdfFactory extends DocumentFactory {
        Document createDocument() {
            return new PdfDocument();
        }
    }
    static class ExcelFactory extends DocumentFactory {
        Document createDocument() {
            return new ExcelDocument();
        }
    }
    public static void main(String[] args) {
        DocumentFactory wordFactory = new WordFactory();
        Document word = wordFactory.createDocument();
        word.open();
```

Supersetid :8013480

```
        DocumentFactory pdfFactory = new PdfFactory();
        Document pdf = pdfFactory.createDocument();
        pdf.open();
        DocumentFactory excelFactory = new ExcelFactory();
        Document excel = excelFactory.createDocument();
        excel.open();
    }
}
```

OUTPUT:



Exercise 3: Implementing the Builder Pattern

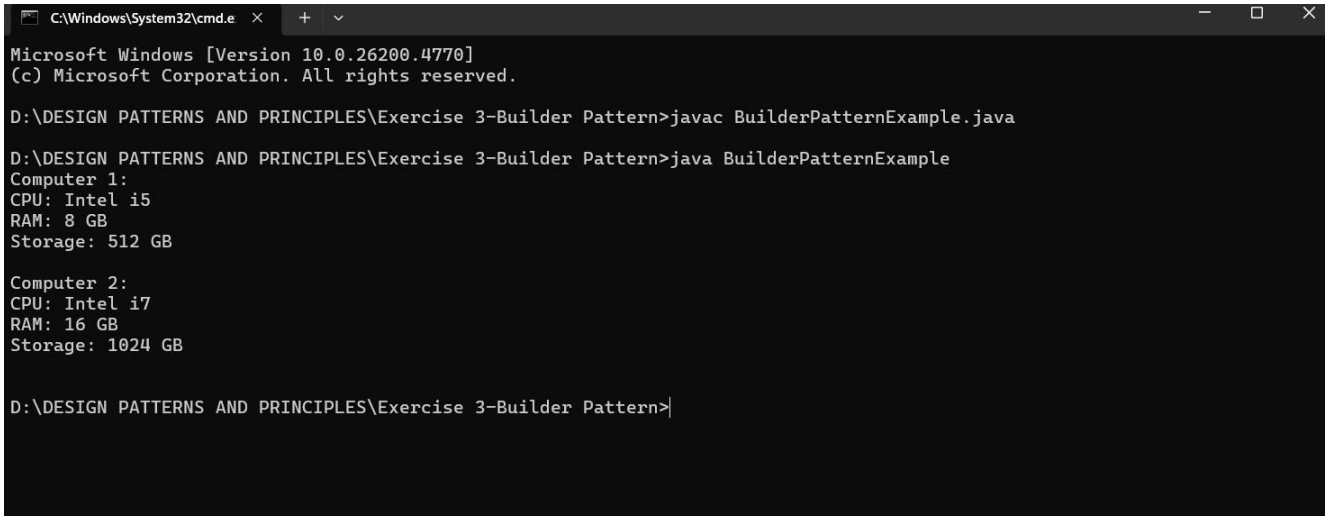
```
public class BuilderPatternExample {
    static class Computer {
        private String cpu;
        private int ram;
        private int storage;
        private Computer(Builder builder) {
            this.cpu = builder.cpu;
            this.ram = builder.ram;
            this.storage = builder.storage;
        }
        static class Builder {
            private String cpu;
            private int ram;
            private int storage;
            public Builder setCpu(String cpu) {
```

Supersetid :8013480

```
            this.cpu = cpu;
            return this;
        }
        public Builder setRam(int ram) {
            this.ram = ram;
            return this;
        }
        public Builder setStorage(int storage) {
            this.storage = storage;
            return this;
        }
        public Computer build() {
            return new Computer(this);
        }
    }
    public void display() {
        System.out.println("CPU: " + cpu);
        System.out.println("RAM: " + ram + " GB");
        System.out.println("Storage: " + storage + " GB");
        System.out.println();
    }
  }
  public static void main(String[] args) {
    Computer computer1 = new Computer.Builder()
            .setCpu("Intel i5")
            .setRam(8)
            .setStorage(512)
            .build();
    Computer computer2 = new Computer.Builder()
            .setCpu("Intel i7")
            .setRam(16)
            .setStorage(1024)
            .build();
    System.out.println("Computer 1:");
    computer1.display();
    System.out.println("Computer 2:");
    computer2.display();
  }
}
```

Supersetid :8013480

OUTPUT:



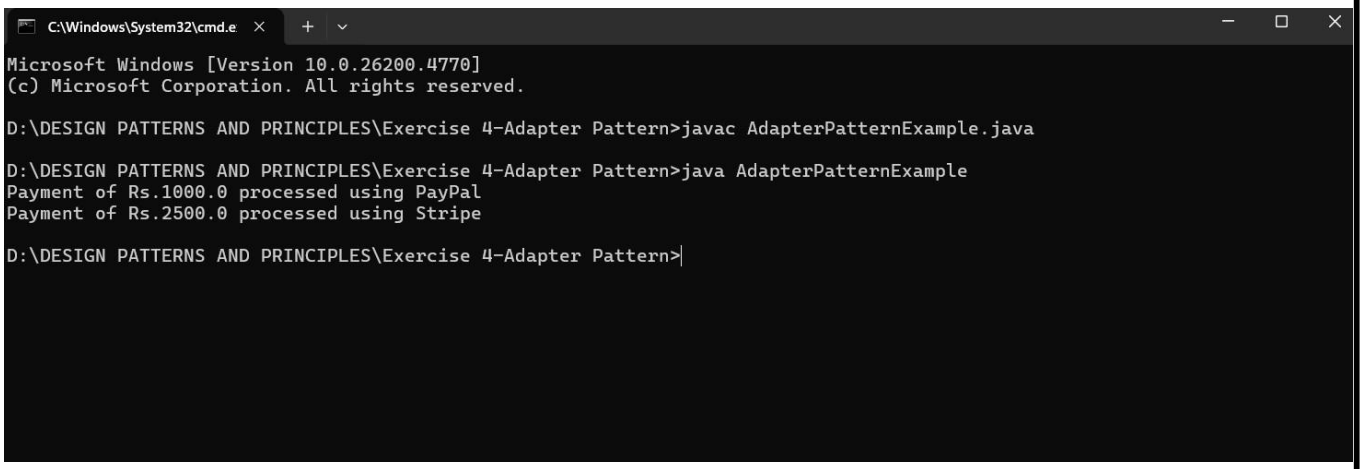
Supersetid :8013480

Exercise 4: Implementing the Adapter Pattern

```
public class AdapterPatternExample {
    interface PaymentProcessor {
        void processPayment(double amount);
    }
    static class PayPalGateway {
        void sendPayment(double amount) {
            System.out.println("Payment of Rs." + amount + " processed using PayPal");
        }
    }
    static class StripeGateway {
        void makeTransaction(double amount) {
            System.out.println("Payment of Rs." + amount + " processed using Stripe");
        }
    }
    static class PayPalAdapter implements PaymentProcessor {
        private PayPalGateway paypal;

        public PayPalAdapter(PayPalGateway paypal) {
            this.paypal = paypal;
        }
        public void processPayment(double amount) {
            paypal.sendPayment(amount);
        }
    }
    static class StripeAdapter implements PaymentProcessor {
        private StripeGateway stripe;
        public StripeAdapter(StripeGateway stripe) {
            this.stripe = stripe;
        }
        public void processPayment(double amount) {
            stripe.makeTransaction(amount);
        }
    }
    public static void main(String[] args) {
        PaymentProcessor payment1 =
                new PayPalAdapter(new PayPalGateway());
        PaymentProcessor payment2 =
                new StripeAdapter(new StripeGateway());
        payment1.processPayment(1000);
        payment2.processPayment(2500);
    }
}
```

Supersetid :8013480

OUTPUT:



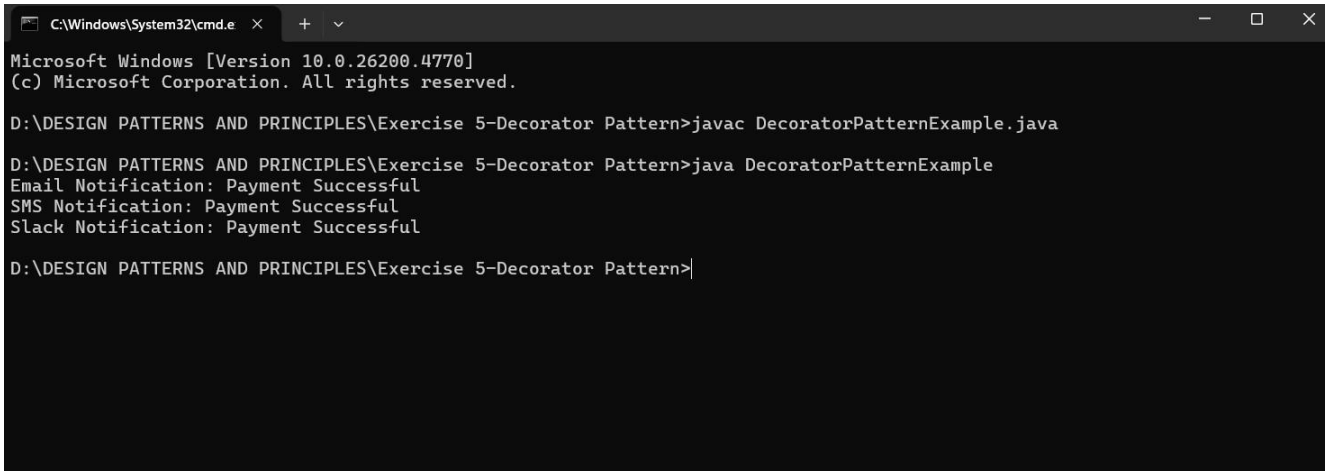
Exercise 5: Implementing the Decorator Pattern

```
public class DecoratorPatternExample {
    interface Notifier {
        void send(String message);
    }
    static class EmailNotifier implements Notifier {
        public void send(String message) {
            System.out.println("Email Notification: " + message);
        }
    }
    static abstract class NotifierDecorator implements Notifier {
        protected Notifier notifier;

        public NotifierDecorator(Notifier notifier) {
            this.notifier = notifier;
        }
        public void send(String message) {
            notifier.send(message);
        }
    }
    static class SMSNotifierDecorator extends NotifierDecorator {

        public SMSNotifierDecorator(Notifier notifier) {
            super(notifier);
        }
        public void send(String message) {
            super.send(message);
            System.out.println("SMS Notification: " + message);
        }
    }
```

Supersetid :8013480

```
    static class SlackNotifierDecorator extends NotifierDecorator {
        public SlackNotifierDecorator(Notifier notifier) {
            super(notifier);
        }
        public void send(String message) {
            super.send(message);
            System.out.println("Slack Notification: " + message);
        }
    }
    public static void main(String[] args) {
        Notifier notifier = new EmailNotifier();
        notifier = new SMSNotifierDecorator(notifier);
        notifier = new SlackNotifierDecorator(notifier);
        notifier.send("Payment Successful");
    }
}
```

OUTPUT:



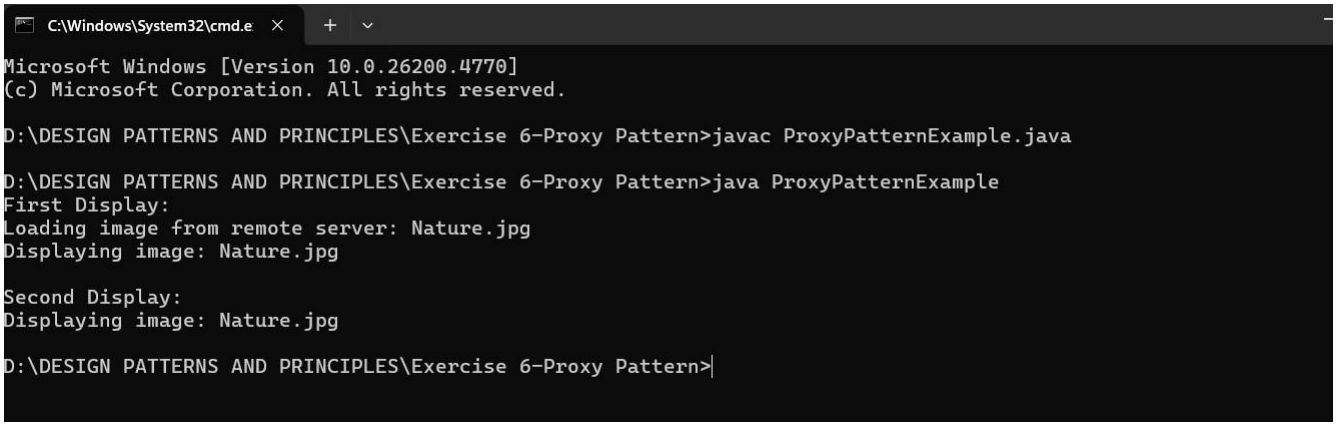
Supersetid :8013480

Exercise 6: Implementing the Proxy Pattern

```
public class ProxyPatternExample {
    interface Image {
        void display();
    }
    static class RealImage implements Image {
        private String fileName;
        public RealImage(String fileName) {
            this.fileName = fileName;
            loadImage();
        }
        private void loadImage() {
            System.out.println("Loading image from remote server: " + fileName);
        }
        public void display() {
            System.out.println("Displaying image: " + fileName);
        }
    }
    static class ProxyImage implements Image {
        private String fileName;
        private RealImage realImage;
        public ProxyImage(String fileName) {
            this.fileName = fileName;
        }
        public void display() {
            if (realImage == null) {
                realImage = new RealImage(fileName); // Lazy initialization
            }
            realImage.display(); // Cached object reused
        }
    }
    public static void main(String[] args) {
        Image image = new ProxyImage("Nature.jpg");
        System.out.println("First Display:");
        image.display();
        System.out.println();
        System.out.println("Second Display:");
        image.display();
    }
}
```

Supersetid :8013480

OUTPUT:



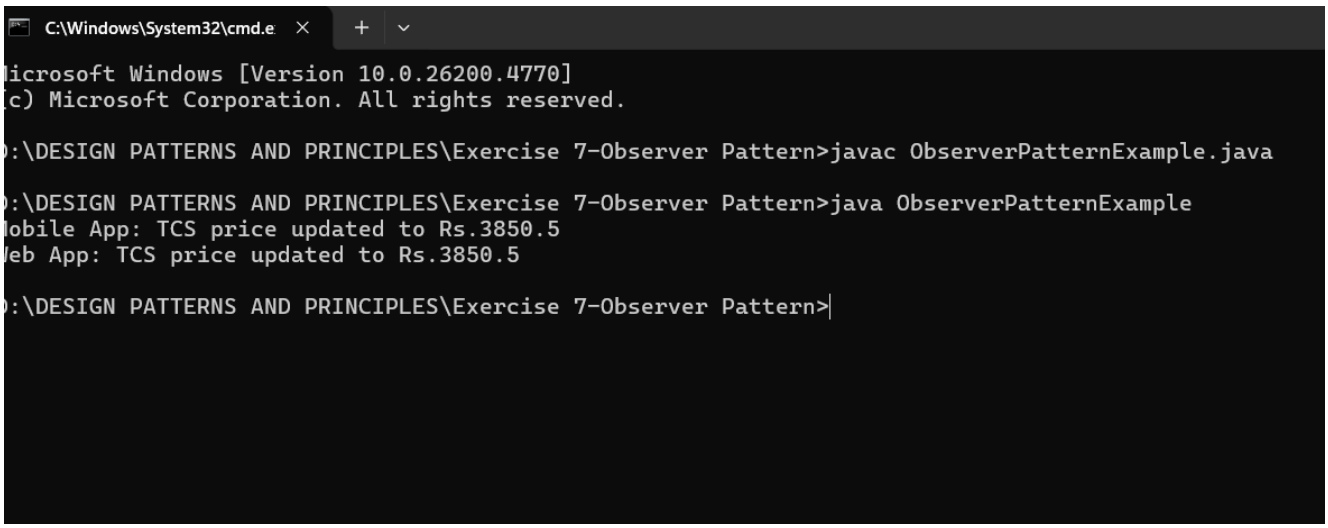
Exercise 7: Implementing the Observer Pattern

```
import java.util.ArrayList;
import java.util.List;
public class ObserverPatternExample {
    interface Observer {
        void update(String stockName, double price);
    }
    interface Stock {
        void registerObserver(Observer observer);
        void removeObserver(Observer observer);
        void notifyObservers();
    }
    static class StockMarket implements Stock {
        private List<Observer> observers = new ArrayList<>();
        private String stockName;
        private double price;
        public void setStockPrice(String stockName, double price) {
            this.stockName = stockName;
            this.price = price;
            notifyObservers();
        }
        public void registerObserver(Observer observer) {
            observers.add(observer);
        }
        public void removeObserver(Observer observer) {
            observers.remove(observer);
        }
        public void notifyObservers() {
            for (Observer observer : observers) {
```

Supersetid :8013480

```
            observer.update(stockName, price);
        }
    }
}
static class MobileApp implements Observer {
    public void update(String stockName, double price) {
        System.out.println("Mobile App: " + stockName +
                           " price updated to Rs." + price);
    }
}
static class WebApp implements Observer {
    public void update(String stockName, double price) {
        System.out.println("Web App: " + stockName +
                           " price updated to Rs." + price);
    }
}
public static void main(String[] args) {
    StockMarket market = new StockMarket();
    Observer mobile = new MobileApp();
    Observer web = new WebApp();
    market.registerObserver(mobile);
    market.registerObserver(web);
    market.setStockPrice("TCS", 3850.50);
}
```

OUTPUT:



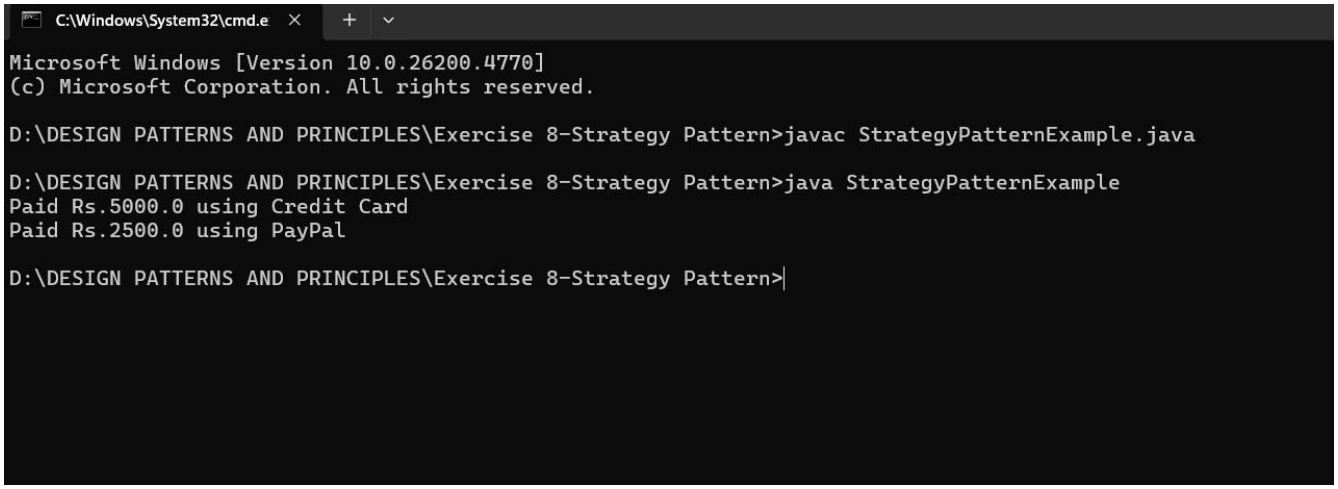
Supersetid :8013480

Exercise 8: Implementing the Strategy Pattern

```
public class StrategyPatternExample {
    interface PaymentStrategy {
        void pay(double amount);
    }
    static class CreditCardPayment implements PaymentStrategy {
        public void pay(double amount) {
            System.out.println("Paid Rs." + amount + " using Credit Card");
        }
    }
    static class PayPalPayment implements PaymentStrategy {
        public void pay(double amount) {
            System.out.println("Paid Rs." + amount + " using PayPal");
        }
    }
    static class PaymentContext {
        private PaymentStrategy strategy;

        public void setPaymentStrategy(PaymentStrategy strategy) {
            this.strategy = strategy;
        }

        public void makePayment(double amount) {
            strategy.pay(amount);
        }
    }
    public static void main(String[] args) {

        PaymentContext context = new PaymentContext();

        context.setPaymentStrategy(new CreditCardPayment());
        context.makePayment(5000);

        context.setPaymentStrategy(new PayPalPayment());
        context.makePayment(2500);
    }
}
```

Supersetid :8013480

OUTPUT:

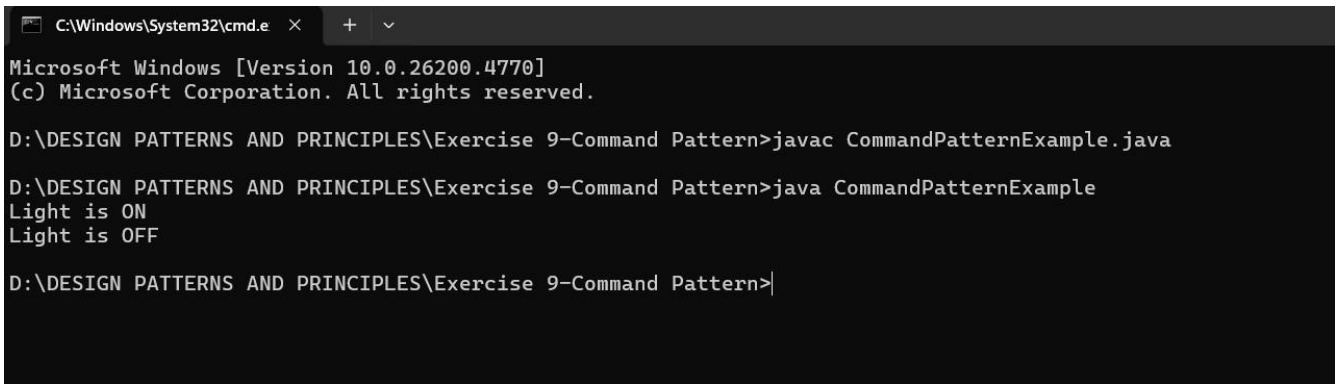


Exercise 9: Implementing the Command Pattern

```
public class CommandPatternExample {
    interface Command {
        void execute();
    }
    static class Light {
        public void turnOn() {
            System.out.println("Light is ON");
        }

        public void turnOff() {
            System.out.println("Light is OFF");
        }
    }
    static class LightOnCommand implements Command {
        private Light light;
        public LightOnCommand(Light light) {
            this.light = light;
        }
        public void execute() {
            light.turnOn();
        }
    }
    static class LightOffCommand implements Command {
        private Light light;
        public LightOffCommand(Light light) {
            this.light = light;
        }
        public void execute() {
            light.turnOff();
```

```
        }
    }
    static class RemoteControl {
        private Command command;
        public void setCommand(Command command) {
            this.command = command;
        }
        public void pressButton() {
            command.execute();
        }
    }
    public static void main(String[] args) {
        Light light = new Light();
        Command onCommand = new LightOnCommand(light);
        Command offCommand = new LightOffCommand(light);
        RemoteControl remote = new RemoteControl();
        remote.setCommand(onCommand);
        remote.pressButton();
        remote.setCommand(offCommand);
        remote.pressButton();
    }
}
```

OUTPUT:



Supersetid :8013480

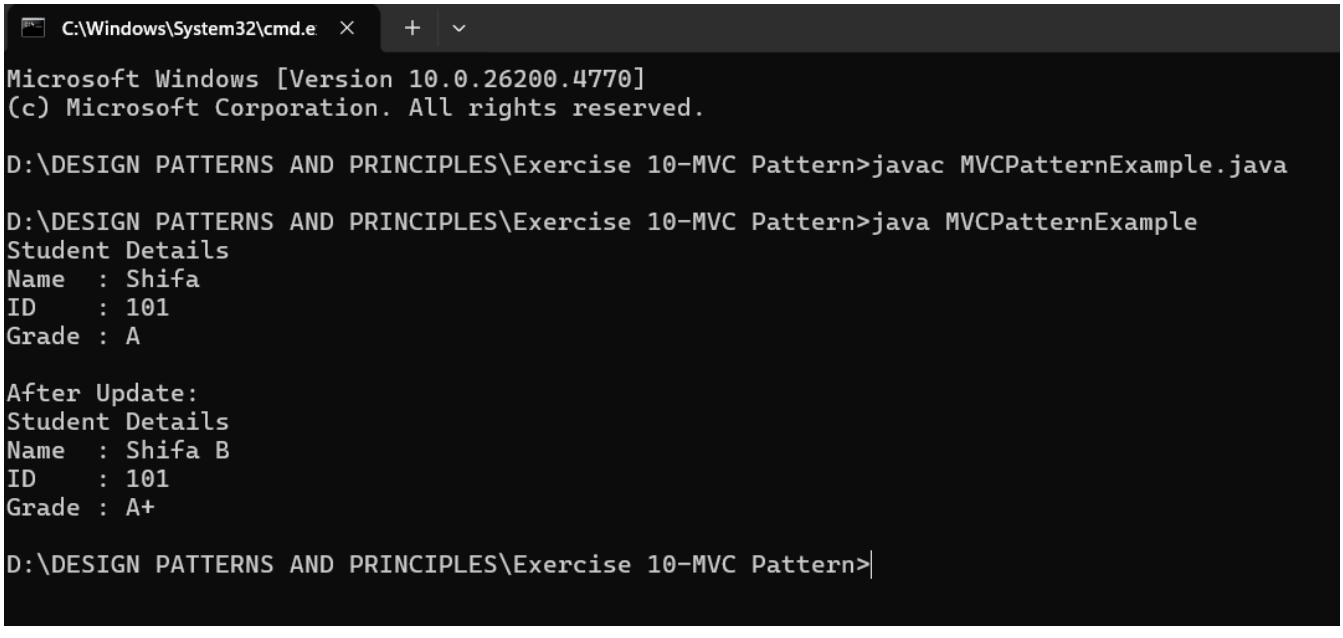
Exercise 10: Implementing the MVC Pattern

```
public class MVCPatternExample {
    static class Student {
        private String name;
        private int id;
        private String grade;
        public Student(String name, int id, String grade) {
            this.name = name;
            this.id = id;
            this.grade = grade;
        }
        public String getName() {
            return name;
        }
        public void setName(String name) {
            this.name = name;
        }
        public int getId() {
            return id;
        }
        public void setId(int id) {
            this.id = id;
        }
        public String getGrade() {
            return grade;
        }
        public void setGrade(String grade) {
            this.grade = grade;
        }
    }
    static class StudentView {
        public void displayStudentDetails(String name, int id, String grade) {
            System.out.println("Student Details");
            System.out.println("Name  : " + name);
            System.out.println("ID     : " + id);
            System.out.println("Grade : " + grade);
        }
    }
    static class StudentController {
        private Student model;
        private StudentView view;
        public StudentController(Student model, StudentView view) {
```

Supersetid :8013480

```
        this.model = model;
        this.view = view;
    }
    public void setStudentName(String name) {
        model.setName(name);
    }
    public void setStudentGrade(String grade) {
        model.setGrade(grade);
    }
    public void updateView() {
        view.displayStudentDetails(
                model.getName(),
                model.getId(),
                model.getGrade());
    }
  }
  public static void main(String[] args) {
    Student student = new Student("Shifa", 101, "A");
    StudentView view = new StudentView();
    StudentController controller =
          new StudentController(student, view);
    controller.updateView();
    System.out.println("\nAfter Update:");
    controller.setStudentName("Shifa B");
    controller.setStudentGrade("A+");
    controller.updateView();
  }
}
```

OUTPUT:

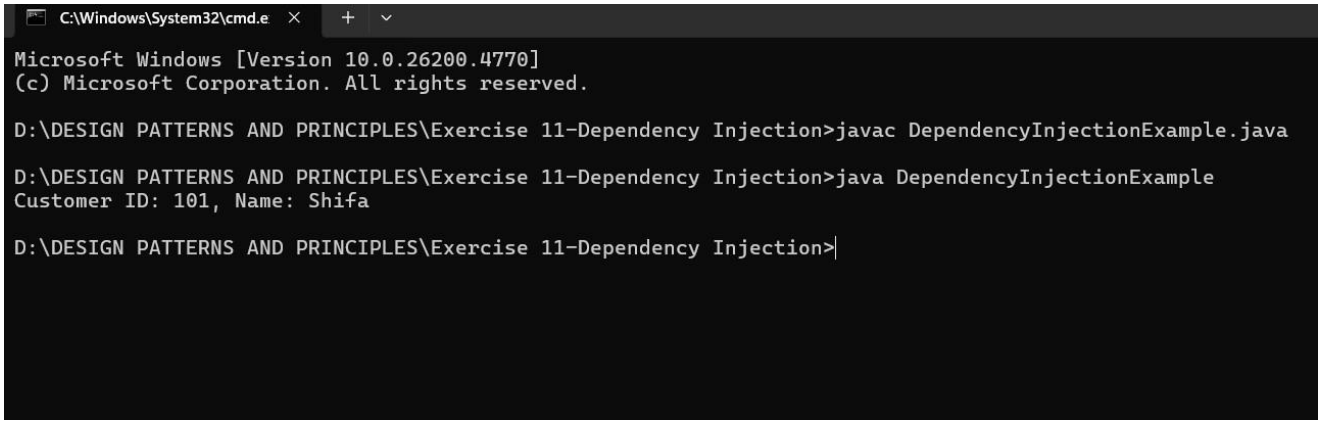


Supersetid :8013480

Exercise 11: Implementing Dependency Injection

```
public class DependencyInjectionExample {
    interface CustomerRepository {
        String findCustomerById(int id);
    }
    static class CustomerRepositoryImpl implements CustomerRepository {
        public String findCustomerById(int id) {
            return "Customer ID: " + id + ", Name: Shifa";
        }
    }
    static class CustomerService {
        private CustomerRepository repository;
        public CustomerService(CustomerRepository repository) {
            this.repository = repository;
        }
        public void getCustomer(int id) {
            System.out.println(repository.findCustomerById(id));
        }
    }
    public static void main(String[] args) {
        CustomerRepository repository =
                new CustomerRepositoryImpl();
        CustomerService service =
                new CustomerService(repository);
        service.getCustomer(101);
    }
}
```

OUTPUT:



Supersetid :8013480